THE STATIC KEYWORD

CS217 – Object Oriented Programming

5.4 STATIC COMPONENTS

5.4.1 The “auto” keyword (1)

- The “**auto**” keyword came from the ancestor of “C++”, the “C” programming language.
- All the variables in your code belong to one of two categories. They are:
  - **automatic variables, created and destroyed automatically** during program execution
  - **static variables, existing continuously** during the whole program execution
- The “C” and “C++” programming languages assume that **all variables are automatic by default** unless they are declared explicitly as static.

5.4.1 The “auto” keyword (1)

- It’s interesting that the “**auto**” keyword is still commonly used in older versions of the “C” language to explicitly mark automatic variables.

```
#include <iostream>
using namespace std;
void fun(void) {
    auto int var = 99;
    cout << "var = " << ++var << endl;
}
int main(void) {
    for(int i = 0; i < 5; i++) fun();
    return 0;
}
```

Output:

```
var = 100
var = 100
var = 100
var = 100
var = 100
```

5.4.1 The “auto” keyword (1)

- The *var* variable is declared inside the *fun* function and is **automatic**.
- We’ve used the “auto” keyword, but the program behavior will **remain the same if you remove** the keyword (try it yourself).
- The variable is **created** each time the fun function is invoked and the variable is **destroyed** each time the function completes its execution.
- This means that, regardless of the number of invocations, the function will always produce the same output.

5.4.2 The “auto” keyword (2)

- Now, **replace the “auto” keyword with “static”**.
- The code looks nearly identical, but its **behavior has been changed radically**.

```
#include <iostream>
using namespace std;
void fun(void) {
    static int var = 99;
    cout << "var = " << ++var << endl;
}
int main(void) {
    for(int i = 0; i < 5; i++) fun();
    return 0;
}
```

Output:

```
var = 100
var = 101
var = 102
var = 103
var = 104
```

5.4.3 Instances of the class

- The class is like a dummy be brought to life when its incarnation (in this case its object) is created.
- Every object created from a particular class is named a class's **instance**.
- From this point of view, each instance of the class is **a separate universe** and has nothing to do with any of the remaining instances.
- All the object's components (fields and functions) are enclosed inside the instance.

5.4.3 Instances of the class

- In consequence, we mustn't use any of the class components until we've created an object of that class.
- The following snippet is wrong and will cause a **compilation error**.

```
#include <iostream>
using namespace std;
class Class {
public:
        int val;
        void print(void) { cout << val << endl; }
};
int main(void) {
        Class::val = 0;  //Error
        Class::print(); //Error
        return 0;
}
```

5.4.4 Static components of the class

- All the rules on the previous slide are true if they refer to the **non-static components** of the class (both fields and functions).
- The “C++” language allows us to define other kinds of components too, they’re called “**static components**”.
- **A static component exists throughout the whole life of the program**.
- Moreover, there is always **only one component** regardless of the number of instances of the class.
- We can say that all the instances **share** the same static components.

5.4.4 Static components of the class

```
#include <iostream>
using namespace std;
class Class {
public:
	static int Static;
	int NonStatic;
	void print(void) {
		cout << "Static = " << ++Static <<
			", NonStatic = " << NonStatic << en
dl;
	}
};
int Class::Static = 0;
int main(void) {
	Class instance1, instance2;
	instance1.NonStatic = 10;
	instance2.NonStatic = 20;
	instance1.print();
	instance2.print();
	return 0;
	}
```

Output:

Static = 1, NonStatic = 10
Static = 2, NonStatic = 20

5.4.4 Static components of the class

- The static variable has to have both
  - **an explicitly expressed separate definition**
  - **a possible initialization, and both must be placed outside the class definition.**
- Another rationale says that the definition has to be separate from the class body because static variables are not actually part of any object.
- In our program, we do this with a line of the code between the *Class* body and the *main* function.
- Removing this line generates an error.

5.4.5 Static class variables (1)

- The static class variables **usually used as counters** of instances of a particular class.
- The next program implements this idea in a very simple way.
  - **We'll increment the static variable *Counter* inside the *Class* constructor and decrement it inside the *Class* destructor**
- Note once again that the *Counter* field is accessed directly when it's being used inside the class and with the "**::**" operator when it's being used outside the class.

5.4.5 Static class variables (1)

```
using namespace std;
class Class {
public:
	static int Counter;
	Class(void) { ++Counter; };
	~Class(void) {
		--Counter;
		if(Counter == 0) cout << "Bye, bye!" << endl;
	};
	void HowMany(void) { cout << Counter << " instances" << endl; }
};
int Class::Counter = 0;
int main(void) {
	Class a;
	Class b;
	cout << Class::Counter << " instances so far" << endl;
	Class c;
	Class d;
	d.HowMany();
	return 0;
	}
```

Output:

2 instances so far
4 instances
Bye, bye!

5.4.6 Static class variables (2)

- **Static** variable can be defined **private** in the class
- This will obviously **prevent direct access** to the variable, but it may be something we want if we want to protect the value against any unauthorized modification.

- Note that any attempts to access the *Counter* variable expressed like this:

~~Class::Counter = 1;~~

are strictly **prohibited**.

5.4.6 Static class variables (2)

```
class Class {
    static int Counter;
public:
    Class(void) {
        ++Counter;
    };
    ~Class(void) { --Counter; if(Counter == 0)
                        cout << "Bye, bye!" << endl;
    };
    void HowMany(void) { cout << Counter << " instances" << endl; }
};
int Class::Counter = 0;
int main(void) {
    Class a;
    Class b;
    b.HowMany();
    Class c;
    Class d;
    d.HowMany();
    return 0;
```

Output:

2 instances
4 instances
Bye, bye!

5.4.7 Static class functions

- It's not only class variables that can be declared as static – **functions** can be declared like this, too.

- The **static function**, like a static variable, may also be accessed (or more precisely, invoked) when no instances of the class have been created.
- Note that the static function may be invoked from inside the class, like this:

```
Class::HowMany();
```

or by using any of the existing instances, like this:

```
b.HowMany();
```

5.4.7 Static class functions

```
class Class {
	static int Counter;
public:
	Class(void) {
		++Counter;
	};
	~Class(void) {
		--Counter;
		if(Counter == 0)
			cout << "Bye, bye!" << endl;
	};
	static void HowMany(void) { cout << Counter << " instances" << endl; }
};
int Class::Counter = 0;
int main(void) {
	Class::HowMany();
	Class a;
	Class b;
	b.HowMany();
	Class c;
	Class d;
	d.HowMany();
	return 0;
```

Output:
0 instances
2 instances
4 instances
Bye, bye!

Static Members of a Class

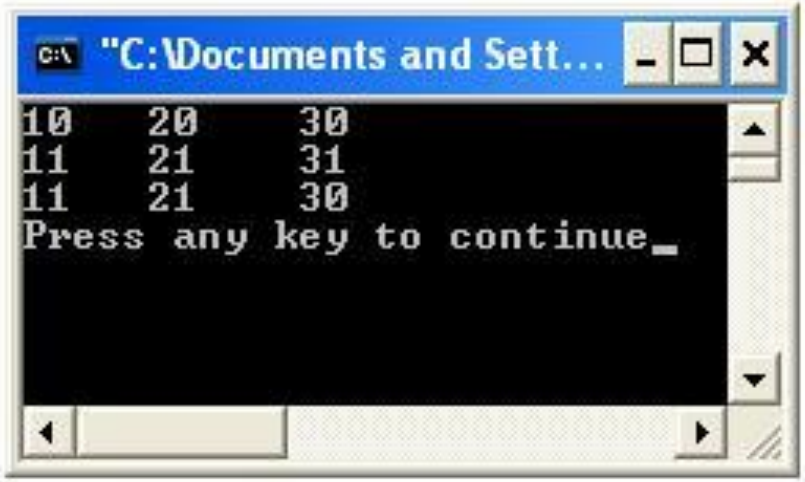
- Use the keyword `static` to declare a function or variable of a class as `static`
- A `public static` function or member of a class can be accessed using the class name and the scope resolution operator
- `static` member variables of a class exist even if no object of that `class` type exists

Static Members of a Class

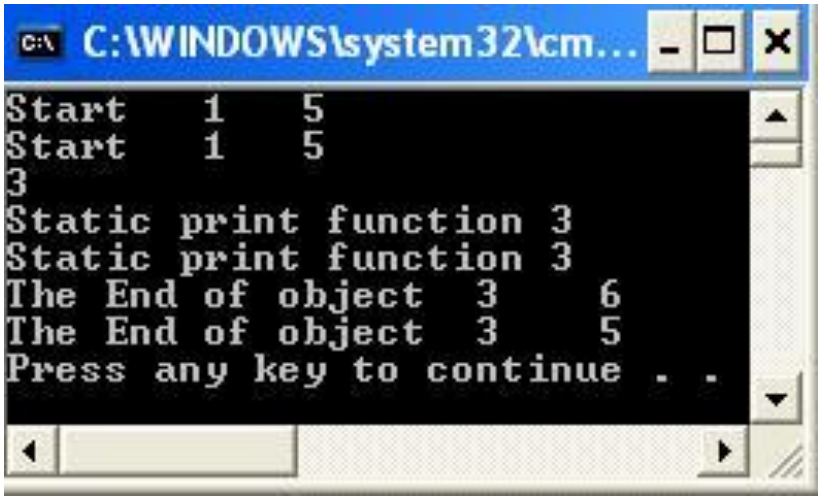
- A `static` member function of a class can only access a static member variable
- A `static` member variable should be initialized out the class braces by using the class name and the scope resolution operator(::)
- `static const` member variable should be initialized within the class

Examples

```
#include <iostream>
#include <string>
using namespace std;

class A
{
    static int y;
public:
    static int x;
    int z;
    void incrementY() {y++;}
    void print() { cout<<x<<"    "<<y<<"    "<<z<<endl;}
    A(){z=30;}
};
int A::x =10;
int A::y=20;


void main()
{
    A obj1;
    obj1.print();
    obj1.x++;
    obj1.incrementY();
    (obj1.z)++;
    obj1.print();
    A obj2;
    obj2.print();
}
```



```
#include <iostream>
using namespace std;;

class A
{
public:
//  static int x=1;    error
    static int x;
    int y;
    static void print();
    ~A(){cout<<"The End of object  "<<x<<"    "<<y<<endl;}
    A(){x=1;y=5;cout<<"Start   "<<x<<"   "<<y<<endl;}
};
int A::x=3;
void A::print()
{
    cout<<"Static print function "<<x<<endl;
    //cout<<y<<endl;     error y is not a static member
}
void main()
{
    A obj1, obj2;
    A::x++;
    obj2.x++;
    obj2.y++;
    cout<<obj1.x<<endl;
    obj2.print();
    A::print();
}
```



5.4.8 Static vs. non-static components

- The coexistence of both static and non-static components within a single class causes some additional issues that we need to take into consideration.
- The **accessed member** may be a data or method, but the **accessing member** is always a function.
  - **Accessed** can be data or a method
  - **Accessing** is always a method.

		accessed component	
		static	non-static
accessing component	static	OK	FAIL
	non-static	OK	OK

5.4.9 Static → static interaction

- The first test program demonstrates a case when **a static function tries to invoke another static function.**
- A case like this is **always possible**, as both functions are available during the entire life of the program.

```
class Test {
public:
    static void funS1(void) { cout << "static" << endl; }
    static void funS2(void) { funS1(); }
};

int main(void) {
    Test object;
    Test::funS2();
    object.funS2();
    return 0;
}
```

Output:
static
static

5.4.11 Static → non-static interaction

- The second test program demonstrates a case when **a static function tries to invoke a non-static function**.
- A case like this is **not possible**, as the function being invoked exists when **and only when** any of the objects which contain this function also exists. The function cannot be successfully accessed without specifying the associated object.

```
class Test {
public:
    void funN1(void) { cout << "non-static" << endl; }
    static void funS1(void) { funN1(); }
};
int main(void) {
    Test object;
    Test::funS1();
    object.funS1();
    return 0;
}
```

Syntax Error
Static member can't invoke non-static member

5.4.13 Non-static → static interaction

- The third test case refers to the situation where **a non-static function invokes a static function**.
- A case like this is **always possible**, as the static function is available before any object has been created.

```
class Test {
public:
	static void funS1(void) { cout << "static" << endl; }
	void funN1(void) { funS1(); }
};

int main(void) {
	Test object;
	object.funN1();
}
```

Output:
static

5.4.15 Static vs. non-static components

- The last remaining option is simple. We don't need to do any experiment to find out the answer to the following question:
  - **Is it possible to invoke a non-static function from within a non-static function?**
- Yes, obviously it is possible. What's more, we've done it many times before.

5.2 A STACK: A VIEW FROM TWO DIFFERENT PERSPECTIVES

5.2.1 Stack aka LIFO (1)

- A **stack** is a structure developed **to store data** in a very specific way.
- Example: A stack of coins would be best here.
  - You can't put or get a coin anywhere else but on the top of the stack.
  - To get the coin on the bottom, you have to remove all the coins that are sitting on top of it.
- The alternative name for a stack is **LIFO: Last In – First Out**.
  - The coin that goes onto the stack last comes off first.

5.2.1 Stack aka LIFO (1)

- A stack is an object to two elementary operations conventionally named
  - "**push**" (when a new element is placed on the top)
  - "**pop**" (when an existing element is taken away from the top).
- Let's try to implement a stack in "C++".
  - It'll be a very simple stack and store *int* values only.
- We'll show you how to do it in two independent approaches: **procedural** and **objective.**
- Let's start with procedural.

5.2.2 Stack aka LIFO (2)

procedural implementation

- First, we have to decide how to store the *int* values that'll arrive onto our stack.
- We suggest using the simplest method and use a vector (array) with no more than 100 values for this job.
- We also assume that the element at index 0 is at the bottom of the stack.
- The stack itself is already declared as:

```
int stack[100];
```

-

5.2.3 Stack pointer

- But the array isn't enough to implement a stack.
- We need a few additional details.
  - **number of elements currently stored** on the stack.
  - This variable is generally called a "**stack pointer**", or **SP** for short.
- Initially, the stack is empty, so the stack pointer should be assigned the value of 0.

```
int sp=0;
```

5.2.4 Push

- We're now ready to define a function that **places a value** onto the stack. Here is what we've supposed:
  - the **name** for the function is "push"
  - the function gets **one parameter** of type *int* (this is the value to be placed onto the stack)
  - the function **returns nothing** (self-explanatory, right?)
  - the function **places the parameter** value into the first free element in the arrayand **increments the SP**
-

```
void push(int value) {
        stack[SP++] = value;
}
```

- The function doesn't check if there's room for the new value.

5.2.5 Pop

- Now it's time for the function to **take a value** off the stack. This is how we imagine it:
  - the **name** of the function is "pop" (we don't want to discover America again)
  - the function **doesn't have any parameters**
  - the function **returns the value** taken from the stack
  - the function **reads the value** from the stack's top and **decrements SP**

```
int pop(void) {
        return stack[--SP];
}
```

- The function doesn't check if there's any element on the stack.

5.2.6 The stack in action

```
#include <iostream>
using namespace std;
int stack[100];
int SP = 0;
void push(int value) { stack[SP++] = value; }
int pop(void) { return stack[--SP]; }
int main(void) {
    push(3);
    push(2);
    push(1);
    cout << pop() << endl;
    cout << pop() << endl;
    cout << pop() << endl;
    return 0;
 }
```

What is the output of this program?!

5.2.7 Pros and cons

- **Disadvantages**
  - two essential variables (stack and SP) are **completely vulnerable;**
    - **anyone can modify them in an uncontrollable way**
  - it may happen that one day you need more than one stack; be prepared to work hard
    - you'll have to create another vector for the stack's storage,
    - another stack pointer for a new vector,
    - probably more *push* and *pop* functions too;
  - it may also happen that you need not only *push* and *pop* functions, but also some other things; you can implement them but try to imagine what'll happen when you have dozens of separately implemented stacks
  - we've used the *int* stack so far, but you may want to use the stacks defined for other types: floats, strings or even arrays and structures; what'll happen then?

5.2.7 Pros and cons

- The objective approach provides solutions for each of these problems.
  - **Encapsulation (chapter 6):** the ability to hide (protect) selected values against unauthorized access
  - when you have a class implementing all the needed stack behaviors, you can produce as many stacks as you want;
  - the ability to enrich the stack with new functions comes from the **inheritance (Chapter 6)**; you can create a new class (or more precisely a **subclass**) which inherits all the existing traits from the superclass and adds some new ones
  - you can create a **template (chapter 8)** which is a **generalized, parameterized class**, ready to materialize itself in many different incarnations; its code can adapt to varying requirements and, for example, create stacks ready to work with other types of data

5.2.8 Stack from scratch (1)

- Let's write a brand new stack implementation from scratch using **objective approach**.
- Stack is a list that contain:
  - A vector (array) as the stack's storage
  - An *int* variable as the stack pointer.
- We wanted to encapsulate both variables and make them inaccessible from the outside world.

```
class Stack {
        int stackstore[100];
        int SP;
};
```

5.2.12 Stack from scratch (5)

- The class is complete now.

```
class Stack {
        private:
                int stackstore[100];
                int SP;
        public:
                Stack(void) { SP = 0; }
                void push(int value);
                int pop(void) { return stackstore[--SP]; }
};
void Stack::push(int value) {
        stackstore[SP++] = value;
}
```

5.2.13 Stack from scratch (6)

- Testing our class:
  - We want to create three objects of the class “Stack”.
  - Next, we want use them in a juggling sort of way.
  - Try to predict the value outputted to the screen.

```
#include <iostream>
using namespace std;
int main(void) {
    Stack   little_stack, another_stack, funny_stack;
    little_stack.push(1);
    another_stack.push(little_stack.pop() + 1);
    funny_stack.push(another_stack.pop() + 2);
    cout << funny_stack.pop() << endl;
return 0; }
```